

Experiment # 13

Implementation of Classifier

Objective

1. To implement classifier in Python

Software

- Python Idle 3.7 or 3.8.5
- Pycharm/SPYDER

Theory

K Nearest Neighbor (KNN) is a very simple, easy to understand, versatile and one of the topmost machine learning algorithms. KNN used in the variety of applications such as finance, healthcare, political science, handwriting detection, image recognition and video recognition. In Credit ratings, financial institutes will predict the credit rating of customers. In loan disbursement, banking institutes will predict whether the loan is safe or risky. In political science, classifying potential voters in two classes will vote or won't vote. KNN algorithm used for both classification and regression problems. KNN algorithm based on feature similarity approach.

The KNN Algorithm

1. Load the data
2. Initialize K to your chosen number of neighbors
3. For each example in the data
 - 3.1 Calculate the distance between the query example and the current example from the data.
 - 3.2 Add the distance and the index of the example to an ordered collection
4. Sort the ordered collection of distances and indices from smallest to largest (in ascending order) by the distances
5. Pick the first K entries from the sorted collection
6. Get the labels of the selected K entries
7. If regression, return the mean of the K labels
8. If classification, return the mode of the K labels

For download data, you can use this url:

`"https://archive.ics.uci.edu/ml/machine-learning-databases/iris/iris.data"`

Some Useful Commands:

The python package that will be needed for the following function is **scikit-image**. You may need to download the .whl file and install it manually if pip doesn't work.

1. To read the csv file `dataset = pd.read_csv(data, labels)`
2. To separate labels and dataset

`X = dataset.iloc[:, :-1].values`
`y = dataset.iloc[:, 4].values`
3. For splitting into train and test dataset
`from sklearn.model_selection import train_test_split`
`X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20)`
4. For classification through KNN
`from sklearn.neighbors import KNeighborsClassifier`
`classifier = KNeighborsClassifier(n_neighbors=5)`
`classifier.fit(X_train, y_train)`
`y_pred = classifier.predict(X_test)`
5. For generating confusion matrix
`from sklearn.metrics import classification_report, confusion_matrix`
`print(confusion_matrix(y_test, y_pred))`
`print(classification_report(y_test, y_pred))`

Lab Task:

1. Apply KNN classifier for fisher iris testing data (70:30 ratio) and calculate its accuracy
2. Also generate its confusion matrix.